

■ Apache Pig — Exam Code Cheatsheet

Module 3 · Pig Latin Quick Reference · All Key Code Patterns

1. LOAD & STORE (Q14, Q25)

LOAD — read CSV from HDFS with schema

```
employees = LOAD 'employees.txt'
USING PigStorage(',') -- delimiter = comma
AS (id:int, name:chararray,
age:int, dept:chararray, salary:int);
```

STORE — write result back to HDFS

```
STORE result INTO 'hdfs://output_path'
USING PigStorage(',');
```

■ Always pair every LOAD with a STORE to persist output.

2. CORE TRANSFORMATIONS (Q2, Q18, Q23, Q24)

FILTER — keep rows matching condition

```
high_sal = FILTER employees BY salary > 60000;
adults = FILTER employees BY age > 18 AND dept == 'IT';
```

FOREACH ... GENERATE — project / compute columns (Q18)

```
-- Select specific columns
info = FOREACH employees GENERATE id, name, dept;

-- Compute new column
comp = FOREACH employees GENERATE id, name,
salary * 1.10 AS new_salary;
```

DISTINCT — remove duplicates (Q23)

```
unique_depts = DISTINCT employees;
```

■ Triggers a MapReduce sort — use only when needed.

LIMIT — cap number of rows (Q24)

```
sample = LIMIT employees 5;
```

■ Great for testing; preview top N rows without full scan.

ORDER BY — sort data

```
sorted = ORDER employees BY salary DESC;
asc_by_age = ORDER employees BY age ASC;
```

3. GROUP & COGROUP (Q10)

GROUP — aggregate one dataset by key

```
grp = GROUP employees BY dept;

-- Output: (HR, {(1,John,...), (4,Sarah,...)})

-- Always follow GROUP with FOREACH for aggregation:
avg_sal = FOREACH grp GENERATE
group AS dept,
```

```
AVG(employees.salary) AS avg_salary,
COUNT(employees) AS emp_count;
DUMP avg_sal;
```

COGROUP — group TWO datasets on same key

```
bonus = LOAD 'bonus.txt' USING PigStorage(',')
AS (id:int, bonus:int);

cg = COGROUP employees BY id, bonus BY id;
-- Output: (1, {(1,John,...)}, {(1,5000)})
```

■ COGROUP != JOIN — it gives nested bags, not flat rows.

4. JOIN OPERATIONS (Q13)

Default JOIN (inner join on common key)

```
emp_bonus = JOIN employees BY id,
bonus BY id;
DUMP emp_bonus; -- flat merged rows
```

```
-- REPLICATED JOIN
-- Use: small table + big table
-- Advantage: map-side only,
-- no shuffle phase
joined = JOIN big BY id,
small BY id
USING 'replicated';
```

```
-- SKEWED JOIN
-- Use: uneven key distribution
-- Advantage: spreads hot keys
-- across multiple reducers
joined = JOIN big BY id,
small BY id
USING 'skewed';
```

5. BUILT-IN FUNCTIONS (Q9, Q15)

String Functions

```
upper_names = FOREACH employees GENERATE id, UPPER(name);
lower_names = FOREACH employees GENERATE id, LOWER(name);
trimmed = FOREACH employees GENERATE TRIM(name);
sub = FOREACH employees GENERATE SUBSTRING(name, 0, 3);
```

Math Functions

```
rounded = FOREACH employees GENERATE ROUND(salary / 1000.0);
absolute = FOREACH employees GENERATE ABS(salary - 60000);
root = FOREACH employees GENERATE SQRT((double)salary);
```

Date & Time Functions (Q15)

```
REGISTER 'piggybank.jar';
dated = FOREACH employees
GENERATE id,
ToDate(CurrentTime()) AS today,
GetYear(ToDate('2025-01-01','yyyy-MM-dd')) AS yr;
```

6. USER-DEFINED FUNCTIONS (UDFs) (Q5)

```
# uppercase.py
@outputSchema('name:chararray')
def to_upper(name):
    if name: return name.upper()
    return None

# --- In Pig script ---
REGISTER 'uppercase.py'
USING jython AS myfuncs;
result = FOREACH employees
GENERATE id,
myfuncs.to_upper(name);
DUMP result;
```

```
// UpperCaseUDF.java
import org.apache.pig.EvalFunc;
import org.apache.pig.data.Tuple;

public class UpperCaseUDF
    extends EvalFunc {
    public String exec(Tuple t)
        throws IOException {
        if (t==null||t.size()==0)
            return null;
        return ((String)t.get(0))
            .toUpperCase();
    }
}

-- In Pig:
REGISTER 'UpperCaseUDF.jar';
r = FOREACH e GENERATE
    UpperCaseUDF(name);
```

■ Python UDF (left) — simpler, no compile step. Java UDF (right) — faster, needs JAR.

7. OPTIMIZATION TECHNIQUES (Q4, Q11)

Technique	Code Example	Why it Helps
Filter Early	<code>FILTER data BY val > 10.0; -- BEFORE joins</code>	Shrinks data early, less shuffle
Projection Pushdown	<code>LOAD ... AS (col1:int, col3:chararray);</code>	Reads only needed columns
Replicated Join	<code>JOIN big BY id, small BY id USING 'replicated';</code>	Map-side join, skips reducers
Skewed Join	<code>JOIN ... USING 'skewed';</code>	Balances hot keys across reducers
Parallelism Control	<code>SET default_parallel 20; GROUP ... PARALLEL 20;</code>	Uses more reducers
Combiner via Algebraic Fn	<code>FOREACH grouped GENERATE COUNT(data), AVG(data.val);</code>	Partial aggregation on mapper
Avoid Nested FOREACH	<code>FOREACH d GENERATE FLATTEN(field);</code>	Flatten instead of nesting
Compress Output	<code>STORE r INTO 'out' USING PigStorage(',', '-schema') COMPRESSION gzip;</code>	Reduces I/O

8. COMPLETE OPTIMIZED SCRIPT (Q7 — Execution Flow)

Full pipeline — Load → Filter → Group → Aggregate → Store

```
-- Step 1: LOAD (with schema)
employees = LOAD 'employees.txt' USING PigStorage(',')
AS (id:int, name:chararray, age:int,
    dept:chararray, salary:int);

-- Step 2: FILTER early (Optimization)
active = FILTER employees BY salary > 0;
```

```

-- Step 3: PROJECT only needed columns (Pushdown)
proj = FOREACH active GENERATE id, name, dept, salary;

-- Step 4: GROUP by department
grp = GROUP proj BY dept;

-- Step 5: AGGREGATE (COUNT + AVG)
summary = FOREACH grp GENERATE
group AS dept,
COUNT(proj) AS headcount,
AVG(proj.salary) AS avg_sal;

-- Step 6: SORT result
final = ORDER summary BY avg_sal DESC;

-- Step 7: STORE output
STORE final INTO 'hdfs://output' USING PigStorage(',');

-- Bonus: DUMP for debugging (never in production)
DUMP final;

```

■ Execution Flow: Script → Parser → Optimizer → Compiler → Execution Engine → HDFS

9. DATA TYPES QUICK REFERENCE (Q3, Q12, Q19)

Category	Type	Example Value	Use For
Primitive	int	25	32-bit whole numbers, IDs
Primitive	long	9876543210L	Large counts, timestamps
Primitive	float	3.14f	Decimal (single precision)
Primitive	double	3.14159265	Decimal (double precision)
Primitive	chararray	'John'	Text / strings
Primitive	bytearray	raw bytes	Default / binary data
Primitive	boolean	true / false	Filter conditions
Complex	Tuple	('John', 25, 'HR')	Single row of fields
Complex	Bag	{ ('John', 25), ... }	Collection of tuples (GROUP output)
Complex	Map	['name' #'John']	Key-value lookup
Atom	Atom	'raja' or 30	Smallest single value

Complex Type Access Syntax

```

-- Tuple field access (by position or name)
t = FOREACH data GENERATE $0, $1; -- position
t = FOREACH data GENERATE name, age; -- name

-- Bag (created by GROUP, accessed in FOREACH)
grp = GROUP data BY city;
cnt = FOREACH grp GENERATE group, COUNT(data);

-- Map (key-value lookup with #)
data = LOAD 'f.txt' AS (info:map[]);
names = FOREACH data GENERATE info#'name';

```

10. PIG vs MapReduce (Q6)

Feature	Apache Pig	MapReduce (Java)
Language	Pig Latin (high-level script)	Java (low-level)
Lines of Code	~5–10 lines	~100+ lines
Complexity	Simple, declarative	Complex, boilerplate-heavy
Optimization	Automatic (Optimizer)	Manual by developer
Data Types	Tuples, Bags, Maps	Key-Value pairs only
Debugging	DUMP in Grunt Shell	Logs / IDE debugger
Best For	ETL, ad-hoc analysis	Custom, low-level jobs